

Lab 6: CUDA Sobel Edge Detection

EEL6763 – Parallel Computer Architecture

Name: Tristan Barber

Partner: Jarrell Reynolds

Part 1: CUDA Sobel Edge Detection Using Global Memory

1.1 Kernel Code

Kernel 1: 1D Global Memory

```
__global__ void sobel_1d(const uint8_t * __restrict__ in, uint8_t * __restrict__ out, int width, int height) {  
  
    int idx = blockDim.x * blockIdx.x + threadIdx.x;  
  
    if (idx >= width * height) return;  
  
    int r = idx / width;  
    int c = idx % width;  
  
    if (r == 0 || r == height - 1 || c == 0 || c == width - 1) {  
        out[idx] = 0;  
        return;  
    }  
  
    int p1 = in[(r-1)*width + (c-1)];  
    int p2 = in[(r-1)*width + c];  
    int p3 = in[(r-1)*width + (c+1)];  
    int q1 = in[r*width + (c-1)];  
    int q3 = in[r*width + (c+1)];  
    int r1 = in[(r+1)*width + (c-1)];  
    int r2 = in[(r+1)*width + c];
```

```

int r3 = in[(r+1)*width + (c+1)];

int qx = (p1 - r1) + 2*(p2 - r2) + (p3 - r3);
int qy = (p1 - p3) + 2*(q1 - q3) + (r1 - r3);
int mag = abs(qx) + abs(qy);
if (mag > 255) mag = 255;

out[idx] = (uint8_t)mag;
}

```

Kernel 2: 2D Global Memory

```

__global__ void sobel_2d(const uint8_t * __restrict__ in, uint8_t * __restrict__ out, int width, int
height) {

    /* Global thread index – 2-D cheatsheet formula */

    int c = blockDim.x * blockIdx.x + threadIdx.x;
    int r = blockDim.y * blockIdx.y + threadIdx.y;

    if (r >= height || c >= width) return;

    /* Skip border pixels */
    if (r == 0 || r == height - 1 || c == 0 || c == width - 1) {
        out[r*width + c] = 0;
        return;
    }

    int p1 = in[(r-1)*width + (c-1)];
    int p2 = in[(r-1)*width + c];
    int p3 = in[(r-1)*width + (c+1)];
    int q1 = in[r*width + (c-1)];
    int q3 = in[r*width + (c+1)];

```

```
int r1 = in[(r+1)*width + (c-1)];  
int r2 = in[(r+1)*width + c];  
int r3 = in[(r+1)*width + (c+1)];  
  
int qx = (p1 - r1) + 2*(p2 - r2) + (p3 - r3);  
int qy = (p1 - p3) + 2*(q1 - q3) + (r1 - r3);  
int mag = abs(qx) + abs(qy);  
if (mag > 255) mag = 255;  
  
out[r*width + c] = (uint8_t)mag;  
}
```

1.2 Part 1 Output Images/ Execution Time

Configuration 1 Execution:

```
==PROF== Disconnected from process 935297
[935297] sobel_cuda@127.0.0.1
sobel_id(const unsigned char *, unsigned char *, int, int) (25000, 1, 1)x(1000, 1, 1), Device 0, CC 8.9, Invocations 1
Section: GPU Speed Of Light Throughput
-----
Metric Name           Metric Unit   Minimum      Maximum      Average
-----
DRAM Frequency        Ghz           6.24         6.24         6.24
SM Frequency          Mhz          827.53       827.53       827.53
Elapsed Cycles        cycle 810,283.00 810,283.00 810,283.00
Memory Throughput     %            34.54        34.54        34.54
DRAM Throughput       %            13.33        13.33        13.33
Duration              us           972.64       972.64       972.64
L1/TEX Cache Throughput %            38.87        38.87        38.87
L2 Cache Throughput   %            15.05        15.05        15.05
SM Active Cycles      cycle 710,440.19 710,440.19 710,440.19
Compute (SM) Throughput %            34.54        34.54        34.54
-----

Section: GPU and Memory Workload Distribution
-----
Metric Name           Metric Unit   Minimum      Maximum      Average
-----
Average DRAM Active Cycles cycle 809,866.67 809,866.67 809,866.67
Total DRAM Elapsed Cycles cycle 36,440,064.00 36,440,064.00 36,440,064.00
Average L1 Active Cycles cycle 710,440.19 710,440.19 710,440.19
Total L1 Elapsed Cycles cycle 46,366,622.00 46,366,622.00 46,366,622.00
Average L2 Active Cycles cycle 790,625.50 790,625.50 790,625.50
Total L2 Elapsed Cycles cycle 19,632,456.00 19,632,456.00 19,632,456.00
Average SM Active Cycles cycle 710,440.19 710,440.19 710,440.19
Total SM Elapsed Cycles cycle 46,366,622.00 46,366,622.00 46,366,622.00
Average SMSP Active Cycles cycle 696,487.57 696,487.57 696,487.57
Total SMSP Elapsed Cycles cycle 185,466,488.00 185,466,488.00 185,466,488.00
-----

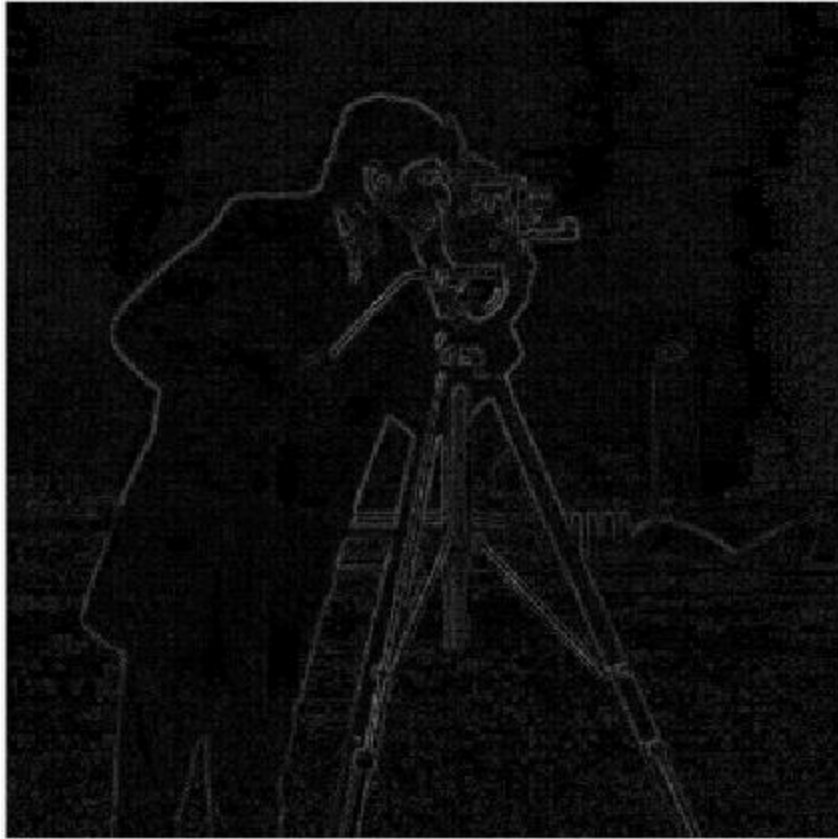
Section: Launch Statistics
-----
Metric Name           Metric Unit   Minimum      Maximum      Average
-----
Block Size            1,000.00     1,000.00     1,000.00
Grid Size             25,000.00    25,000.00    25,000.00
Registers Per Thread  register/thread 20.00        20.00        20.00
Shared Memory Configuration Size Kbyte 8.19         8.19         8.19
Driver Shared Memory Per Block Kbyte/block 1.02         1.02         1.02
Dynamic Shared Memory Per Block byte/block 0.00         0.00         0.00
Static Shared Memory Per Block byte/block 0.00         0.00         0.00
# SMs                 SM           58.00        58.00        58.00
Stack Size            1,024.00     1,024.00     1,024.00
Threads              thread 25,000,000.00 25,000,000.00 25,000,000.00
# TPCs               TPC          29.00        29.00        29.00
Uses Green Context    0.00         0.00         0.00
Waves Per SM          431.03       431.03       431.03
-----

Section: Occupancy
-----
Metric Name           Metric Unit   Minimum      Maximum      Average
-----
Block Limit SM        block 24.00        24.00        24.00
Block Limit Registers  block 2.00         2.00         2.00
Block Limit Shared Mem block 8.00         8.00         8.00
Block Limit Warps      block 1.00         1.00         1.00
Theoretical Active Warps per SM warp 32.00        32.00        32.00
Theoretical Occupancy % 66.67        66.67        66.67
Achieved Occupancy     % 60.39        60.39        60.39
Achieved Active Warps Per SM warp 28.99       28.99        28.99
-----

Note: The shown averages are calculated as the arithmetic mean of the metric values after the evaluation of the metrics for each individual kernel launch.
If aggregating across varying launch configurations (like shared memory, cache config settings), the arithmetic mean can be misleading and looking at the individual results is recommended.
This output mode is backwards compatible to the per-kernel summary output of nvprof

Total program wall-clock time: 1.81528032 sec
[tbarber@login8 Part1]$ █
```

Configuration 1 Output:



Configuration 2 Execution:

```
==PROF== Disconnected from process 1956257
[1956257] sobel_cuda@127.0.0.1
sobel_2d(const unsigned char *, unsigned char *, int, int) (50, 500, 1)x(100, 10, 1), Device 0, CC 8.9, Invocations 1
Section: GPU Speed Of Light Throughput
-----
Metric Name           Metric Unit   Minimum   Maximum   Average
-----
DRAM Frequency        Ghz           6.24      6.24      6.24
SM Frequency           Mhz          811.90    811.90    811.90
Elapsed Cycles         cycle 652,383.00 652,383.00 652,383.00
Memory Throughput      %            47.19     47.19     47.19
DRAM Throughput        %            16.44     16.44     16.44
Duration               us           794.72    794.72    794.72
L1/TEX Cache Throughput %            54.89     54.89     54.89
L2 Cache Throughput    %            19.51     19.51     19.51
SM Active Cycles       cycle 553,340.21 553,340.21 553,340.21
Compute (SM) Throughput %            47.19     47.19     47.19
-----

Section: GPU and Memory Workload Distribution
-----
Metric Name           Metric Unit   Minimum   Maximum   Average
-----
Average DRAM Active Cycles cycle 815,741.33 815,741.33 815,741.33
Total DRAM Elapsed Cycles cycle 29,773,824.00 29,773,824.00 29,773,824.00
Average L1 Active Cycles cycle 553,340.21 553,340.21 553,340.21
Total L1 Elapsed Cycles cycle 37,325,718.00 37,325,718.00 37,325,718.00
Average L2 Active Cycles cycle 611,735.12 611,735.12 611,735.12
Total L2 Elapsed Cycles cycle 15,734,112.00 15,734,112.00 15,734,112.00
Average SM Active Cycles cycle 553,340.21 553,340.21 553,340.21
Total SM Elapsed Cycles cycle 37,325,718.00 37,325,718.00 37,325,718.00
Average SMSP Active Cycles cycle 541,200.94 541,200.94 541,200.94
Total SMSP Elapsed Cycles cycle 149,302,872.00 149,302,872.00 149,302,872.00
-----

Section: Launch Statistics
-----
Metric Name           Metric Unit   Minimum   Maximum   Average
-----
Block Size            1,000.00     1,000.00   1,000.00
Grid Size             25,000.00    25,000.00  25,000.00
Registers Per Thread  register/thread 20.00      20.00      20.00
Shared Memory Configuration Size Kbyte        8.19       8.19       8.19
Driver Shared Memory Per Block Kbyte/block   1.02       1.02       1.02
Dynamic Shared Memory Per Block byte/block     0.00       0.00       0.00
Static Shared Memory Per Block byte/block     0.00       0.00       0.00
# SMs                 SM            58.00      58.00      58.00
Stack Size            1,024.00     1,024.00   1,024.00
Threads               thread 25,000,000.00 25,000,000.00 25,000,000.00
# TPCs                29.00        29.00      29.00
Uses Green Context    0.00         0.00       0.00
Waves Per SM          431.03       431.03     431.03
-----

Section: Occupancy
-----
Metric Name           Metric Unit   Minimum   Maximum   Average
-----
Block Limit SM        block        24.00     24.00     24.00
Block Limit Registers  block        2.00      2.00      2.00
Block Limit Shared Mem block         8.00      8.00      8.00
Block Limit Warps     block        1.00      1.00      1.00
Theoretical Active Warps per SM warp          32.00     32.00     32.00
Theoretical Occupancy %            66.67     66.67     66.67
Achieved Occupancy    %            57.04     57.04     57.04
Achieved Active Warps Per SM warp          27.38     27.38     27.38
-----

Note: The shown averages are calculated as the arithmetic mean of the metric values after the evaluation of the metrics for each individual kernel launch.
If aggregating across varying launch configurations (like shared memory, cache config settings), the arithmetic mean can be misleading and looking at the individual results is recommended instead.
This output mode is backwards compatible to the per-kernel summary output of nvprof

Total program wall-clock time: 2.05266304 sec
[tbarber@login8 Part1]$
```

Configuration 2:



1.3 Performance Table

Grid Configuration	Block Configuration	Kernel Time (s)	Wall Time (s)
(25000,1,1)	(1000,1,1)	0.000973	1.815
(50,500,1)	(100,10,1)	0.000795	2.053

1.4 Comparison with Lab 4

Implementation	Execution Time (s)
CPU	2.037
CPU + MPI	1.014
GPU (Global Memory)	1.815

1.5 Analysis

The CUDA implementation using global memory demonstrated improved performance compared to the serial CPU implementation, reducing execution time from approximately 2.037 seconds to 1.815 seconds. However, the MPI implementation achieved the best performance at approximately 1.014 seconds.

While Configuration 2 achieved a lower kernel execution time, Configuration 1 produced the lowest overall wall clock time, indicating that total execution time is influenced not only by kernel computation but also by memory transfer and execution overhead.

Kernel execution time was significantly smaller than total wall time, demonstrating that host-device memory transfers contributed substantially to total execution time.

The best configuration for global memory was:

Grid = (25000,1,1), Block = (1000,1,1)

Part 2: CUDA Sobel Edge Detection Using Shared Memory

2.1 Kernel Code

Kernel 1: 1D Shared Memory

```
__global__ void sobel_1d_shared(const uint8_t * __restrict__ in, uint8_t * __restrict__ out, int width,
int height) {

    int tid = blockDim.x * blockIdx.x + threadIdx.x;

    int r = tid / width;

    int c = tid % width;

    int tx = threadIdx.x;

    int bw = blockDim.x;

    int smem_w = bw + 2;

    extern __shared__ uint8_t smem[];

    smem[0 * smem_w + (tx + 1)] = (r - 1 >= 0 && c >= 0 && c < width) ? in[(r-1)*width + c] : 0;
    smem[1 * smem_w + (tx + 1)] = (r >= 0 && r < height && c >= 0 && c < width) ? in[r*width + c] : 0;
    smem[2 * smem_w + (tx + 1)] = (r + 1 < height && c >= 0 && c < width) ? in[(r+1)*width + c] : 0;

    if (tx == 0) {
```

```

smem[0 * smem_w + 0] = (r - 1 >= 0 && c - 1 >= 0) ? in[(r-1)*width + (c-1)] : 0;
smem[1 * smem_w + 0] = (r >= 0 && r < height && c - 1 >= 0) ? in[r*width + (c-1)] : 0;
smem[2 * smem_w + 0] = (r + 1 < height && c - 1 >= 0) ? in[(r+1)*width + (c-1)] : 0;
}

if (tx == bw - 1) {
    smem[0 * smem_w + (bw + 1)] = (r - 1 >= 0 && c + 1 < width) ? in[(r-1)*width + (c+1)] : 0;
    smem[1 * smem_w + (bw + 1)] = (r >= 0 && r < height && c + 1 < width) ? in[r*width + (c+1)] : 0;
    smem[2 * smem_w + (bw + 1)] = (r + 1 < height && c + 1 < width) ? in[(r+1)*width + (c+1)] : 0;
}

__syncthreads();

if (tid >= width * height) return;
if (r == 0 || r == height - 1 || c == 0 || c == width - 1) {
    out[tid] = 0;
    return;
}

int sc = tx + 1;
int p1 = smem[0 * smem_w + (sc - 1)];
int p2 = smem[0 * smem_w + sc];
int p3 = smem[0 * smem_w + (sc + 1)];
int q1 = smem[1 * smem_w + (sc - 1)];
int q3 = smem[1 * smem_w + (sc + 1)];
int r1 = smem[2 * smem_w + (sc - 1)];
int r2 = smem[2 * smem_w + sc];
int r3 = smem[2 * smem_w + (sc + 1)];

```

```

int qx = (p1 - r1) + 2*(p2 - r2) + (p3 - r3);
int qy = (p1 - p3) + 2*(q1 - q3) + (r1 - r3);
int mag = abs(qx) + abs(qy);
if (mag > 255) mag = 255;

out[tid] = (uint8_t)mag;
}

```

Kernel 2: 2D Shared Memory

```

__global__ void sobel_2d_shared(const uint8_t * __restrict__ in, uint8_t * __restrict__ out, int width,
int height) {

    int c = blockDim.x * blockIdx.x + threadIdx.x;
    int r = blockDim.y * blockIdx.y + threadIdx.y;
    int tx = threadIdx.x;
    int ty = threadIdx.y;
    int bw = blockDim.x;
    int bh = blockDim.y;
    int smem_w = bw + 2;

    extern __shared__ uint8_t smem[];

    smem[(ty + 1) * smem_w + (tx + 1)] = (r >= 0 && r < height && c >= 0 && c < width) ? in[r*width + c] :
0;

    if (ty == 0)
        smem[0 * smem_w + (tx + 1)] = (r - 1 >= 0 && c >= 0 && c < width) ? in[(r-1)*width + c] : 0;
    if (ty == bh - 1)
        smem[(bh + 1) * smem_w + (tx + 1)] = (r + 1 < height && c >= 0 && c < width) ? in[(r+1)*width + c] :
0;

    if (tx == 0)

```

```

    smem[(ty + 1) * smem_w + 0] = (r >= 0 && r < height && c - 1 >= 0) ? in[r*width + (c-1)] : 0;
    if (tx == bw - 1)
        smem[(ty + 1) * smem_w + (bw + 1)] = (r >= 0 && r < height && c + 1 < width) ? in[r*width + (c+1)] :
0;

    if (tx == 0 && ty == 0)
        smem[0 * smem_w + 0] = (r - 1 >= 0 && c - 1 >= 0) ? in[(r-1)*width + (c-1)] : 0;
    if (tx == bw - 1 && ty == 0)
        smem[0 * smem_w + (bw + 1)] = (r - 1 >= 0 && c + 1 < width) ? in[(r-1)*width + (c+1)] : 0;
    if (tx == 0 && ty == bh - 1)
        smem[(bh + 1) * smem_w + 0] = (r + 1 < height && c - 1 >= 0) ? in[(r+1)*width + (c-1)] : 0;
    if (tx == bw - 1 && ty == bh - 1)
        smem[(bh + 1) * smem_w + (bw + 1)] = (r + 1 < height && c + 1 < width) ? in[(r+1)*width + (c+1)] :
0;

    __syncthreads();

    if (r >= height || c >= width) return;
    if (r == 0 || r == height - 1 || c == 0 || c == width - 1) {
        out[r*width + c] = 0;
        return;
    }

    int sr = ty + 1;
    int sc = tx + 1;
    int p1 = smem[(sr - 1) * smem_w + (sc - 1)];
    int p2 = smem[(sr - 1) * smem_w + sc];
    int p3 = smem[(sr - 1) * smem_w + (sc + 1)];
    int q1 = smem[sr * smem_w + (sc - 1)];

```

```
int q3 = smem[sr * smem_w + (sc + 1)];
int r1 = smem[(sr + 1) * smem_w + (sc - 1)];
int r2 = smem[(sr + 1) * smem_w + sc];
int r3 = smem[(sr + 1) * smem_w + (sc + 1)];

int qx = (p1 - r1) + 2*(p2 - r2) + (p3 - r3);
int qy = (p1 - p3) + 2*(q1 - q3) + (r1 - r3);
int mag = abs(qx) + abs(qy);
if (mag > 255) mag = 255;

out[r*width + c] = (uint8_t)mag;
}
```

2.2 Part 2 Output Images/Execution Time

Configuration 1 Execution:

```
==PROF== Disconnected from process 1956256
[1956256] sobel_cuda_shared@127.0.0.1
sobel_id shared(const unsigned char *, unsigned char *, int, int) (25000, 1, 1)x(1000, 1, 1), Device 0, CC 8.9, Invocations 1
Section: GPU Speed Of Light Throughput
-----
Metric Name           Metric Unit           Minimum           Maximum           Average
-----
DRAM Frequency        Ghz                   6.24              6.24              6.24
SM Frequency           Mhz                   794.94            794.94            794.94
Elapsed Cycles         cycle 1,025,011.00     1,025,011.00     1,025,011.00
Memory Throughput      %                     44.14             44.14             44.14
DRAM Throughput        %                     9.69              9.69              9.69
Duration               ms                    1.29              1.29              1.29
L1/TEX Cache Throughput %                     48.09             48.09             48.09
L2 Cache Throughput    %                     8.73              8.73              8.73
SM Active Cycles       cycle 939,789.29       939,789.29       939,789.29
Compute (SM) Throughput %                     44.14             44.14             44.14
-----

Section: GPU and Memory Workload Distribution
-----
Metric Name           Metric Unit           Minimum           Maximum           Average
-----
Average DRAM Active Cycles cycle 779,965.33       779,965.33       779,965.33
Total DRAM Elapsed Cycles cycle 48,307,200.00     48,307,200.00     48,307,200.00
Average L1 Active Cycles cycle 939,789.29       939,789.29       939,789.29
Total L1 Elapsed Cycles cycle 59,389,474.00     59,389,474.00     59,389,474.00
Average L2 Active Cycles cycle 971,064.83       971,064.83       971,064.83
Total L2 Elapsed Cycles cycle 27,399,432.00     27,399,432.00     27,399,432.00
Average SM Active Cycles cycle 939,789.29       939,789.29       939,789.29
Total SM Elapsed Cycles cycle 59,389,474.00     59,389,474.00     59,389,474.00
Average SMSP Active Cycles cycle 927,007.37       927,007.37       927,007.37
Total SMSP Elapsed Cycles cycle 237,557,896.00     237,557,896.00     237,557,896.00
-----

Section: Launch Statistics
-----
Metric Name           Metric Unit           Minimum           Maximum           Average
-----
Block Size            1,000.00              1,000.00          1,000.00
Grid Size             25,000.00             25,000.00          25,000.00
Registers Per Thread  register/thread        20.00              20.00              20.00
Shared Memory Configuration Size Kbyte 8.19                 8.19                 8.19
Driver Shared Memory Per Block Kbyte/block 1.02                1.02                1.02
Dynamic Shared Memory Per Block Kbyte/block 3.01                3.01                3.01
Static Shared Memory Per Block byte/block 0.00                 0.00                 0.00
# SMs                  SM 58.00                58.00                58.00
Stack Size            1,024.00              1,024.00          1,024.00
Threads               thread 25,000,000.00     25,000,000.00     25,000,000.00
# TPCs                29.00                 29.00                29.00
Uses Green Context     0.00                  0.00                 0.00
Waves Per SM          431.03                431.03              431.03
-----

Section: Occupancy
-----
Metric Name           Metric Unit           Minimum           Maximum           Average
-----
Block Limit SM        block 24.00              24.00              24.00
Block Limit Registers block 2.00              2.00              2.00
Block Limit Shared Mem block 2.00              2.00              2.00
Block Limit Warps     block 1.00              1.00              1.00
Theoretical Active Warps per SM warp 32.00              32.00              32.00
Theoretical Occupancy % 66.67                66.67              66.67
Achieved Occupancy    % 64.60                64.60              64.60
Achieved Active Warps Per SM warp 31.01              31.01              31.01
-----

Note: The shown averages are calculated as the arithmetic mean of the metric values after the evaluation of the metrics for each individual kernel launch.
If aggregating across varying launch configurations (like shared memory, cache config settings), the arithmetic mean can be misleading and looking at the individual results is recommended instead.
This output mode is backwards compatible to the per-kernel summary output of nvprof

grid=(25000,1,1) block=(1000,1,1)

Total program wall-clock time: 2.06248081 sec
```

Configuration 1 Output:



Configuration 2 Execution:

```
==PROF== Disconnected from process 3072460
[3072460] sobel_cuda_shared@127.0.0.1
sobel_2d_shared(const unsigned char *, unsigned char *, int, int) (50, 500, 1)x(100, 10, 1), Device 0, CC 8.9, Invocations 1
Section: GPU Speed Of Light Throughput
-----
Metric Name           Metric Unit      Minimum      Maximum      Average
-----
DRAM Frequency        Ghz              6.24         6.24         6.24
SM Frequency          Mhz             794.95       794.95       794.95
Elapsed Cycles        cycle 1,169,964.00 1,169,964.00 1,169,964.00
Memory Throughput     %               35.39        35.39        35.39
DRAM Throughput       %               9.02         9.02         9.02
Duration              ms              1.47         1.47         1.47
L1/TEX Cache Throughput %               38.26        38.26        38.26
L2 Cache Throughput   %               10.38        10.38        10.38
SM Active Cycles      cycle 1,081,129.10 1,081,129.10 1,081,129.10
Compute (SM) Throughput %               35.39        35.39        35.39
-----

Section: GPU and Memory Workload Distribution
-----
Metric Name           Metric Unit      Minimum      Maximum      Average
-----
Average DRAM Active Cycles cycle 829,378.67 829,378.67 829,378.67
Total DRAM Elapsed Cycles cycle 55,139,328.00 55,139,328.00 55,139,328.00
Average L1 Active Cycles cycle 1,081,129.10 1,081,129.10 1,081,129.10
Total L1 Elapsed Cycles cycle 67,778,678.00 67,778,678.00 67,778,678.00
Average L2 Active Cycles cycle 1,029,448.67 1,029,448.67 1,029,448.67
Total L2 Elapsed Cycles cycle 29,391,960.00 29,391,960.00 29,391,960.00
Average SM Active Cycles cycle 1,081,129.10 1,081,129.10 1,081,129.10
Total SM Elapsed Cycles cycle 67,778,678.00 67,778,678.00 67,778,678.00
Average SMSP Active Cycles cycle 1,067,359.89 1,067,359.89 1,067,359.89
Total SMSP Elapsed Cycles cycle 271,114,712.00 271,114,712.00 271,114,712.00
-----

Section: Launch Statistics
-----
Metric Name           Metric Unit      Minimum      Maximum      Average
-----
Block Size            1,000.00        1,000.00        1,000.00
Grid Size             25,000.00       25,000.00       25,000.00
Registers Per Thread  register/thread  23.00          23.00          23.00
Shared Memory Configuration Size Kbyte          8.19           8.19           8.19
Driver Shared Memory Per Block Kbyte/block    1.02           1.02           1.02
Dynamic Shared Memory Per Block Kbyte/block    1.22           1.22           1.22
Static Shared Memory Per Block byte/block      0.00           0.00           0.00
# SMs                 SM              58.00          58.00          58.00
Stack Size            1,024.00        1,024.00        1,024.00
Threads               thread 25,000,000.00 25,000,000.00 25,000,000.00
# TPCs                29.00           29.00           29.00
Uses Green Context    0.00            0.00            0.00
Waves Per SM          431.03          431.03          431.03
-----

Section: Occupancy
-----
Metric Name           Metric Unit      Minimum      Maximum      Average
-----
Block Limit SM        block 24.00         24.00         24.00
Block Limit Registers  block 2.00          2.00          2.00
Block Limit Shared Mem block 3.00         3.00          3.00
Block Limit Warps     block 1.00         1.00          1.00
Theoretical Active Warps per SM warp 32.00         32.00         32.00
Theoretical Occupancy % 66.67         66.67         66.67
Achieved Occupancy    % 64.78        64.78         64.78
Achieved Active Warps Per SM warp 31.09        31.09         31.09
-----

Note: The shown averages are calculated as the arithmetic mean of the metric values after the evaluation of the metrics for each individual kernel launch.
If aggregating across varying launch configurations (like shared memory, cache config settings), the arithmetic mean can be misleading and looking at the individual results is recommended instead.
This output mode is backwards compatible to the per-kernel summary output of nvprof

grid=(50,500,1) block=(100,10,1)

Total program wall-clock time: 1.97395708 sec
[tbarber@login8 Part2]$
```

Configuration 2:



2.3 Performance Table

Grid Configuration	Block Configuration	Kernel Time (s)	Wall Time (s)
(25000,1,1)	(1000,1,1)	0.00129	2.062
(50,500,1)	(100,10,1)	0.00147	1.974

2.4 Comparison with Lab 4

Implementation	Execution Time (s)
CPU	2.037
CPU + MPI	1.014
GPU (Shared Memory)	1.974

2.5 Analysis

The shared memory implementation was expected to improve performance due to reduced global memory access latency. However, the results indicate that shared memory did not outperform the global memory implementation in terms of overall wall clock time.

Although kernel execution time improved in some configurations, the additional overhead associated with loading data into shared memory and thread synchronization reduced the overall benefit.

The best configuration for shared memory was:

Grid = (50,500,1), Block = (100,10,1)

Global vs Shared Memory Comparison

3.1 Performance Summary

Implementation	Execution Time (s)
CPU	2.037
CPU + MPI	1.014
GPU (Global Memory)	1.815
GPU (Shared Memory)	1.974

3.2 Discussion

Although GPUs provide massive parallelism, the MPI implementation achieved the best performance in this lab. This is likely due to the relatively simple computation of the Sobel operator and the overhead associated with GPU memory transfers and kernel launches.

Shared memory did not provide a performance advantage in this implementation, as the cost of loading data into shared memory and synchronizing threads outweighed its benefits.

The global memory implementation performed better due to simpler memory access patterns and lower overhead.

This demonstrates that performance depends not only on hardware capability but also on how well the algorithm maps to the architecture.

4. Conclusion

In this lab, CUDA-based implementations of the Sobel edge detection algorithm were developed using both global and shared memory.

Key findings:

- MPI implementation achieved the best performance in this lab

- GPU implementations did not outperform MPI due to overhead costs
- Global memory performed better than shared memory in this case
- Proper grid and block configuration is critical for performance